

Universidad Rey Juan Carlos

Escuela Técnica Superior
Ingeniería Informática

Asignatura: Procesadores de Lenguajes

Autores:

Diego Iglesias Peña

Sergio Muñoz Laureiro

Índice

1. Introducción
2. Reglas Léxicas
3. Transformación de la gramática a LL(1)
4. Notificación y recuperación de errores
5. Explicación de los casos de prueba
6. Anexo: conjuntos directores

1. Introducción

El presente documento detalla el diseño e implementación de los analizadores léxico y sintáctico para un lenguaje de programación de cálculo científico, desarrollado como primera fase del traductor orientado a C. Se ha implementado tanto la funcionalidad obligatoria como las ampliaciones opcionales correspondientes a esta fase (constantes en nuevas bases, constantes lógicas y sentencias de control de flujo ampliadas).

2. Reglas Léxicas

De acuerdo con las especificaciones, el analizador léxico presenta ciertas expresiones regulares que requieren atención especial por su complejidad:

- Constantes literales (Cadenas): Se ha diseñado la regla *STRING_CONST* para soportar delimitadores simples (') y dobles ("). El aspecto más relevante es el soporte para el escape de comillas internas mediante su duplicación (ej. "" dentro de una cadena delimitada por comillas dobles), lo cual requiere un manejo preciso de los grupos de negación en ANTLR.
- Constantes reales: La regla *NUM_REAL_CONST* engloba el formato de punto fijo, el formato exponencial y el mixto. La complejidad se encontraba en hacer opcionales ciertas partes (como el signo del exponente) sin generar ambigüedades con el signo de resta u otras constantes enteras.
- Constantes numéricas opcionales: Para evitar solapamientos léxicos, las bases binaria, octal y hexadecimal se han restringido para que exijan el prefijo estricto (b, o, z) seguido obligatoriamente de comillas simples envolviendo los dígitos válidos para cada base.

3. Transformación de la Gramática a LL(1)

Con el fin de asegurar un análisis sintáctico que sea eficiente, la gramática inicial fue sometida a un proceso de normalización.

3.1. Eliminación de la recursividad por la izquierda

La recursividad por la izquierda es incompatible con los analizadores que trabajan en forma descendente, ya que ocasiona un bucle infinito al intentar expandir el símbolo no terminal antes de procesar cualquier token.

- Ejemplo en la práctica (*sentlist*):

Originalmente, *sentlist* se define de forma recursiva izquierda:

```
sentlist ::= sentlist sent | sent.
```

Se ha transformado mediante la introducción de un símbolo auxiliar (*sentlistp*) para convertirla en recursiva por la derecha:

```
sentlist : sent sentlistp ;
```

```
sentlistp: sent sentlistp | /*Lambda*/;
```

- Ejemplo en expresiones (*exp*):

Se ha aplicado el mismo principio a las expresiones aritméticas, separando los operandos (*factor*) de la continuación de la operación (*expp*):

```
exp:factor exp;
```

```
expp:op factor exp | /*Lambda*/ ;
```

3.2. Factorización por la izquierda

Este proceso se aplica cuando varias producciones de un mismo no terminal comparten un prefijo, lo que impide que el parser decida el camino a seguir con un solo token.

- Ejemplo: La sentencia IF

En algunos lenguajes, el IF puede ser simple o tener un bloque THEN/ELSE:

```
sent ::= ... | "IF" "(" expcond ")" sent | "IF" "("  
expcond ")" "THEN" sentlist "ENDIF" | "IF"  
"(" expcond ")" "THEN" sentlist "ELSE" sentlist "ENDIF"  
| ...
```

Para evitar el conflicto, se ha extraído el prefijo común("IF" "(" expcond ")") :

```
sent: ... | 'IF' '(' expcond ')' sentif | ...  
sentif : sent // IF simple
```

```
    | 'THEN' sentlist sentthen; // IF con bloque, aquí se vuelve a  
aplicar factorización por la izquierda
```

```
sentthen : 'ENDIF'  
    | 'ELSE' sentlist 'ENDIF';
```

Aquí, la decisión sobre el ELSE se pospone hasta haber procesado el cuerpo del THEN.

3.3. Resolución de conflictos y Lookahead

Se han identificado situaciones donde la diferencia semántica requeriría una gramática mucho más complicada para ser estrictamente LL(1). En estas circunstancias se ha optado por:

1. Ambigüedad en Declaraciones (*codproc* y *codfun*): Al definir subprogramas, el token tipo (*INTEGER*, *REAL*, etc.) aparece tanto en la lista de parámetros (*dec_s_paramlist*) como en las declaraciones locales (*dcllist*). Dado que *dec_s_paramlist* puede ser vacío, el parser ve un tipo y no sabe a qué sección pertenece sin mirar más adelante. Tratamos de corregir esto, pero por la complejidad de este cambio, decidimos dejarlo así, resultando en LL(2). Además, en la regla *codfun* tenemos también un problema

parecido con *sentlist* e *IDENT*. En este caso, cuando encuentra *IDENT* no sabe si pertenece a *sentlist* o es el último *IDENT*. Tratamos de implementar una solución a esto con una nueva regla que tratase de evitar el problema, pero no fue posible ya que o bien acabamos en el mismo problema que en primer caso (el de *dcllist*) o bien permitía algunas estructuras que no debían ser permitidas. También se consideró eliminar un tramo ya que pensamos que podía ser una redundancia, pero esto también resultaba en algo no esperado, por lo que decidimos dejarlo en LL(*).

Se confía en la capacidad de ANTLR4 para resolver estos conflictos, manteniendo la gramática legible y fiel al estándar del lenguaje solicitado.

4. Gestión de Errores: Notificación y Recuperación

La robustez de un procesador de lenguajes se evalúa por su habilidad para manejar entradas incorrectas sin interrumpir de manera brusca el análisis. En esta práctica, se han introducido mecanismos específicos para mejorar la interacción con el usuario y la durabilidad del parser.

4.1. Implementación de la Notificación de Errores

Para mejorar la claridad del feedback, se ha creado un Listener personalizado extendiendo la clase `BaseErrorListener` de ANTLR. Este componente permite interceptar las excepciones antes de que se muestren por la salida estándar, eliminando los manejadores por defecto mediante `removeErrorListeners()`. Al detectar una inconsistencia, el sistema produce una notificación completa que incluye:

- Localización exacta: Se informa de la línea y columna precisas donde el token no coincide con las reglas de la gramática.
- Contexto del fallo: El mensaje indica el token inesperado y, en base a los conjuntos de Directores de la gramática LL(1) transformada, sugiere qué tipo de estructura se esperaba en ese punto.

Para ejecutar el caso de prueba se usará el siguiente comando en la terminal:

```
java -jar miPrograma.jar PRUEBAS/CasoIncorrecto.txt
```

-Resultado en terminal(pruebaNotificacionError.txt):

```
Iniciando análisis sintáctico...

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 2 | Posición: 0
>> Detalle: mismatched input '::' expecting ';'
>> Token que falló: '::'
#####

Análisis finalizado.
```

4.2. Resolución de la Recuperación de Errores

Con el objetivo de permitir que el análisis continúe tras detectar un fallo y poder reportar todos los errores del fichero en una sola pasada, se han aplicado las siguientes estrategias de recuperación:

- Sincronización en Modo Pánico: Ante errores complejos en estructuras iterativas como las listas de sentencias (*sentlist*) o declaraciones (*dcllist*), el parser entra en un estado de búsqueda. Utiliza tokens de sincronización (como ; , *END*) para saltar el fragmento de código erróneo y retomar el análisis en un punto seguro.
- Recuperación en Línea (Mismatched Token Exception): El parser es capaz de realizar reparaciones simples sobre la marcha. Si se detecta la falta de un token predecible, como un paréntesis en una expresión condicional o un delimitador, ANTLR intenta realizar una "inserción virtual" o un "borrado" del token sobrante para validar la regla actual y proseguir con la siguiente instrucción sin detener el flujo de ejecución.

En el siguiente punto se verá un ejemplo en el que se demuestra la recuperación de errores.

5. Explicación de los Casos de Prueba

Para validar el correcto funcionamiento del analizador léxico y sintáctico, se han diseñado 8 casos de prueba (4 correctos y 4 erróneos) que ponen al límite las restricciones de la gramática. En los casos de prueba correctos se explicará brevemente la parte de la gramática que está probando junto a su árbol sintáctico completo sin errores, mientras que en los casos incorrectos a parte de lo anterior se mostrará el/los errores junto a una breve justificación del error.

Cabe recalcar que, los mensajes de error estarán en el formato de la notificación de error implementada en el apartado anterior(4.1).

La visualización y verificación de los árboles sintácticos generados se ha implementado mediante la herramienta ANTLR Tool Preview que ofrece IntelliJ donde al poner en la terminal o importar un script del caso de prueba se genera el árbol sintáctico completo, indicando el nombre de la regla y el valor que tiene en función del caso de prueba pasado y marcado en rojo los errores. Las capturas de los árboles que se muestran a continuación han sido obtenidas con esta herramienta, todo ello para una mejor ilustración y sencillez a la hora de visualizar esta memoria.

En el proyecto se adjuntan los 8 archivos de casos de prueba utilizados en este trabajo en la carpeta PRUEBA.

5.1. Casos de Prueba Correctos

Caso Correcto 1- Estructura básica, prueba el esqueleto del programa, la declaración de constantes y variables (con inicialización), y el uso del IF simple de una sola línea validando que acepta sentencias generales y no solo asignaciones.

-Código:

```
PROGRAM test_parte_basico;

INTEGER :: x, y = 10;

REAL, PARAMETER :: PI = 3.141592;

INTERFACE

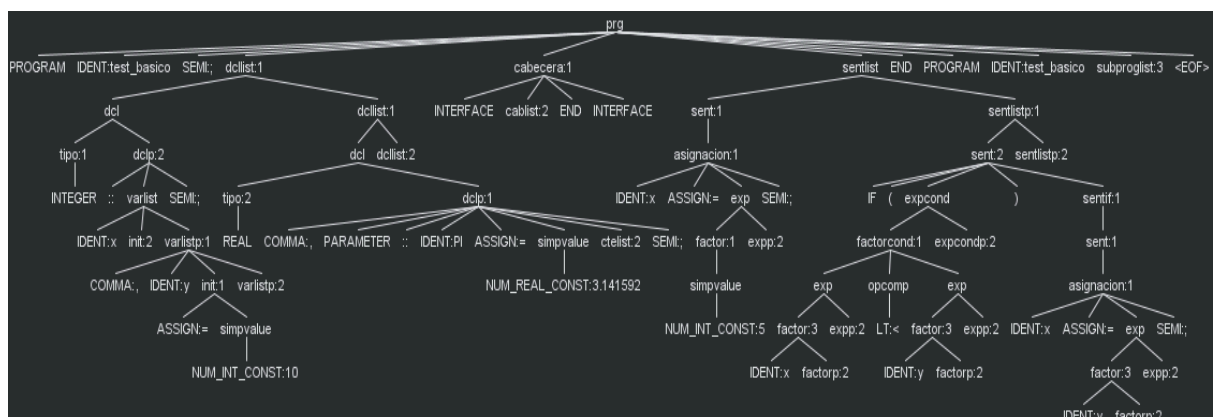
END INTERFACE

x = 5;

IF (x < y) x = y;

END PROGRAM test_parte_basico
```

-Árbol Sintáctico:



Caso Correcto 2- Sentencias de control y parte opcional (bases y condicionales). Verifica las sentencias DO WHILE, SELECT CASE y el uso correcto de las constantes en nuevas bases (como la hexadecimal z'...' y binaria b'...') además de los valores lógicos.

-Código:

```
PROGRAM test_parte_opcional;

INTEGER :: mascara;

CHARACTER(10) :: estado;

INTERFACE

END INTERFACE

mascara = z'A3F';

DO WHILE (.TRUE. .AND. (mascara /= 0))

    SELECT CASE (mascara)

        CASE (b'1010')

            estado = "ACTIVO";

        CASE DEFAULT

            mascara = 0;

    END SELECT

ENDDO

END PROGRAM test_parte_opcional
```

[illegible]

-Código:

```
PROGRAM Climatizador;

INTEGER :: temp_actual;

INTERFACE

    SUBROUTINE enfriar_rapido

    END SUBROUTINE enfriar_rapido

    SUBROUTINE set_temp(t)

        INTEGER, INTENT(IN) t;

    END SUBROUTINE set_temp

END INTERFACE
```

```

temp_actual = 30;

IF (temp_actual > 25) CALL enfriar_rapido;

CALL set_temp(22);

END PROGRAM Climatizador

SUBROUTINE enfriar_rapido

    CALL activar_compresor;

END SUBROUTINE enfriar_rapido

SUBROUTINE set_temp(t)

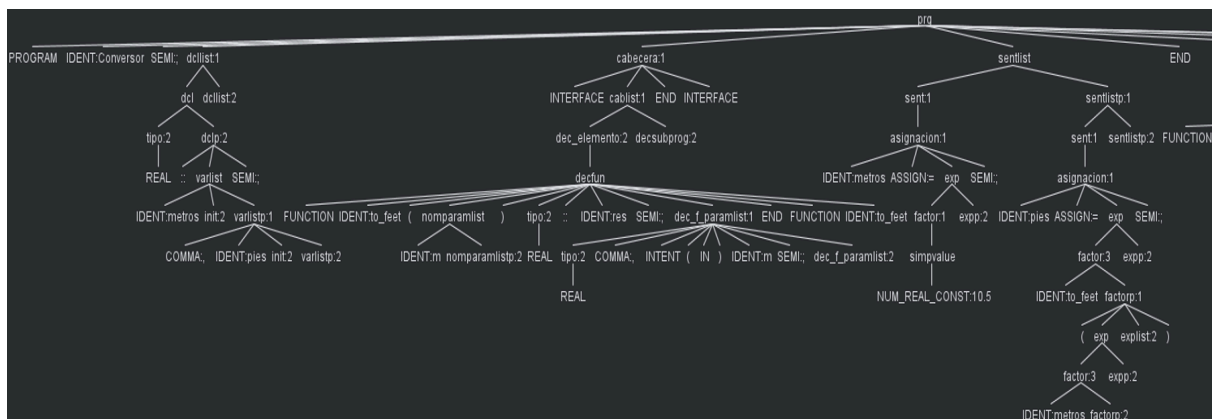
    INTEGER, INTENT(IN) t;

    temp_actual = t;

END SUBROUTINE set_temp

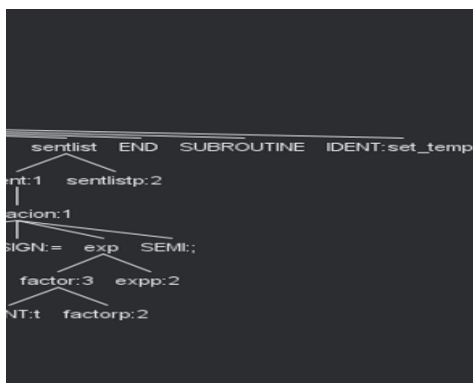
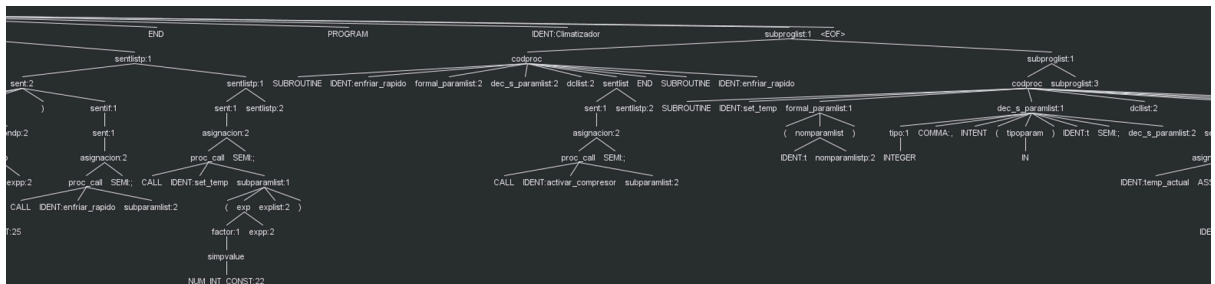
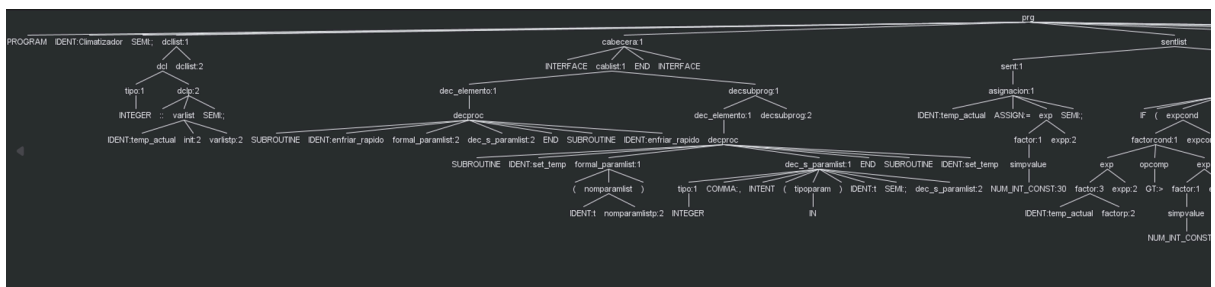
```

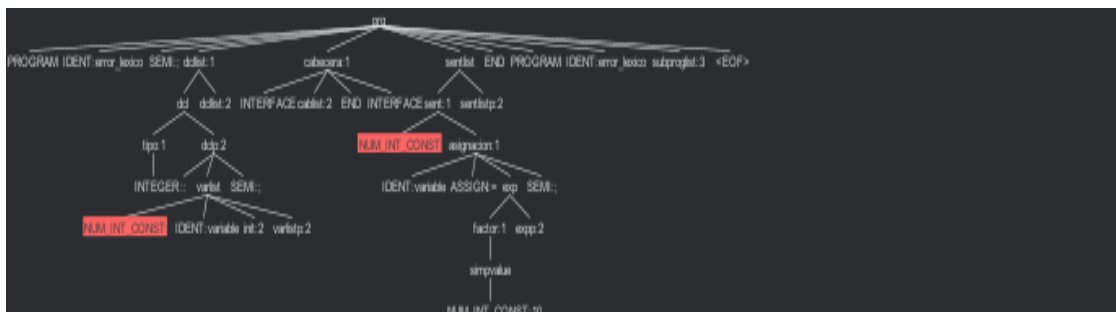
-Árbol Sintáctico:




```
END FUNCTION to_feet
```

-Árbol Sintáctico:





-Error:

```
Iniciando análisis del archivo: PRUEBAS/CasoIncorrecto1.txt...
ERROR LÉXICO (Símbolo no reconocido)
Línea 3:12 -> token recognition error at: '_'
-----

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 3 | Posición: 11
>> Detalle: extraneous input '1' expecting IDENT
>> Token que falló: '1'
#####

ERROR LÉXICO (Símbolo no reconocido)
Línea 6:1 -> token recognition error at: '_'
-----

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 6 | Posición: 0
>> Detalle: extraneous input '1' expecting {'IF', 'DO', 'SELECT', 'CALL', IDENT}
>> Token que falló: '1'
#####

Análisis finalizado.
```

Los mensajes token recognition error at: '_' confirman que el analizador léxico no reconoce el guión bajo como inicio de token en esa posición, y el extraneous input '1' indica que el dígito es completamente inesperado donde la gramática exige un identificador válido.

Caso Erróneo 2- Lista de sentencias vacía: Presenta un bloque THEN de una sentencia IF sin cuerpo. La regla *sentlist* ha sido diseñada para no permitir cadenas vacías directas, exigiendo sintácticamente al menos una instrucción válida (*sent*) en su interior.

-Código:

```
PROGRAM error_sintactico;

INTEGER :: x = 0;

INTERFACE

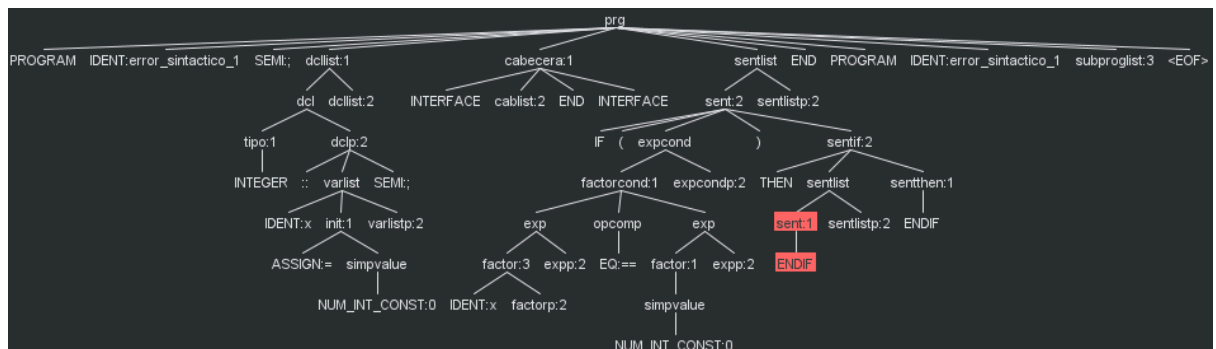
END INTERFACE
```

```
IF (x == 0) THEN
```

```
ENDIF
```

```
END PROGRAM error_sintactico
```

-Árbol Sintáctico:



-Error:

```

Iniciando análisis del archivo: PRUEBAS/CasoIncorrecto2.txt...

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 7 | Posición: 0
>> Detalle: mismatched input 'ENDIF' expecting {'IF', 'DO', 'SELECT', 'CALL', IDENT}
>> Token que falló: 'ENDIF'
#####

Análisis finalizado.
```

El mensaje (mismatched input 'ENDIF') confirma que el parser esperaba al menos una sentencia válida antes de cerrar el bloque.

Caso Erróneo 3- Ruptura de la jerarquía de bloques (IF anidado): El código presenta una estructura de control IF-THEN anidada donde se intenta cerrar el bloque externo antes que el interno, o se introduce una sentencia en un lugar donde el parser, por su naturaleza LL(1), ya ha "decidido" que el bloque ha terminado.

-Código:

```
PROGRAM error_anidamiento;

INTEGER :: a = 10, b = 20;

INTERFACE

END INTERFACE

IF (a > 0) THEN

    IF (b > 0) THEN

        a = a + 1;

    ENDIF

ELSE

    a = 0;

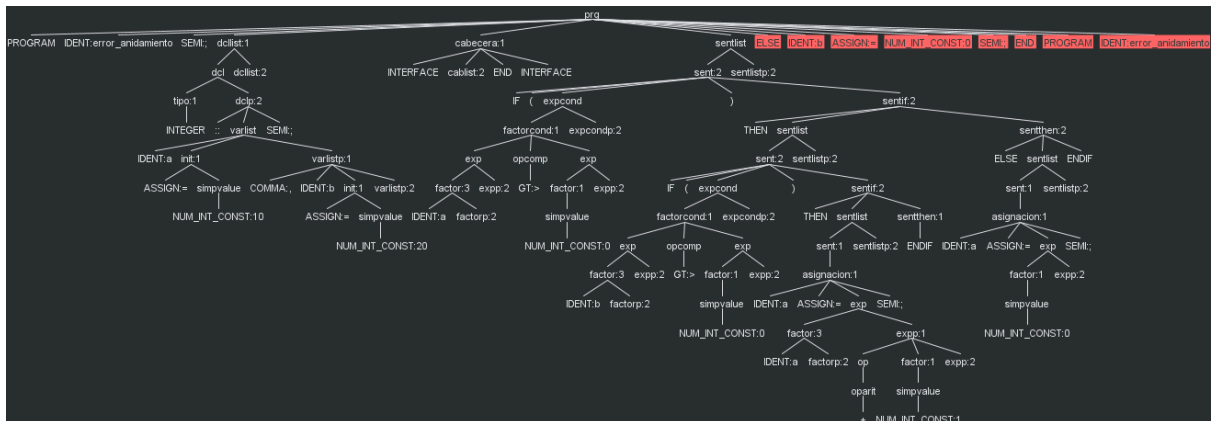
ENDIF

ELSE

    b = 0;

END PROGRAM error_anidamiento
```

-Árbol Sintáctico:



-Error:

```
Iniciando análisis del archivo: PRUEBAS/CasoIncorrecto3.txt...

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 14 | Posición: 0
>> Detalle: mismatched input 'ELSE' expecting {'+', '-', '*', '/', ';'}
>> Token que falló: 'ELSE'
#####

Análisis finalizado.
```

El mensaje mismatched input 'ELSE' confirma que el parser ya no está dentro de una estructura condicional y espera o una nueva sentencia o el final del programa (END PROGRAM), pero nunca un ELSE huérfano.

Caso Erróneo 4 - Ausencia de retorno estricto: La función no sitúa la asignación final de su propio valor (*IDENT = exp ;*) justo antes de la cláusula *END FUNCTION*. La presencia de otra sentencia (*CALL log_error;*) colocada de manera posterior rompe directamente la estructura de la producción *codfun*.

- Código:

```
PROGRAM error_estructural;

INTEGER :: res;

INTERFACE

    FUNCTION operar (a) INTEGER :: operar;

    INTEGER, INTENT(IN) a;

    END FUNCTION operar

END INTERFACE

res = operar(5);

END PROGRAM error_estructural

FUNCTION operar (a) INTEGER :: operar;

INTEGER, INTENT(IN) a;

IF (a > 0) THEN

    operar = a * 2;

ELSE

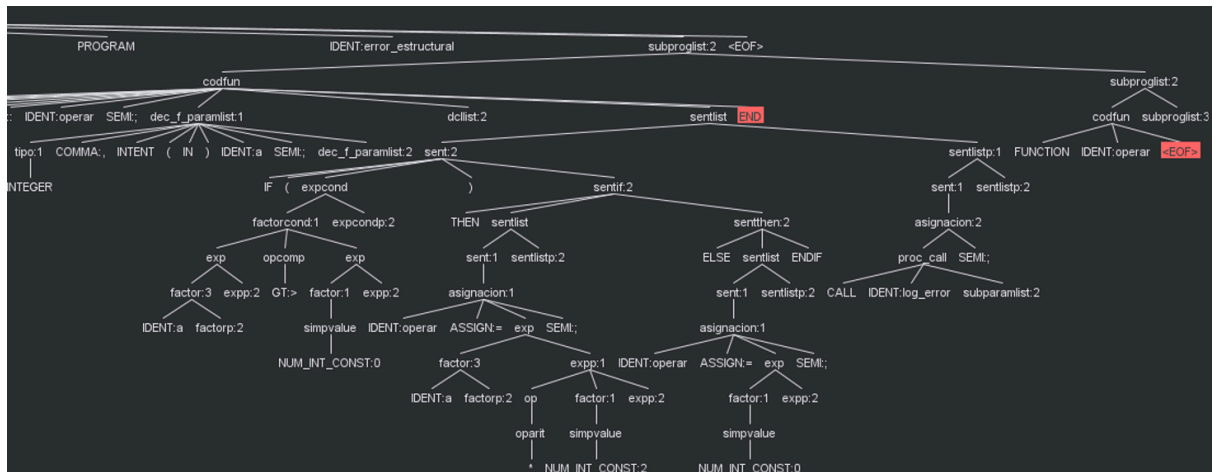
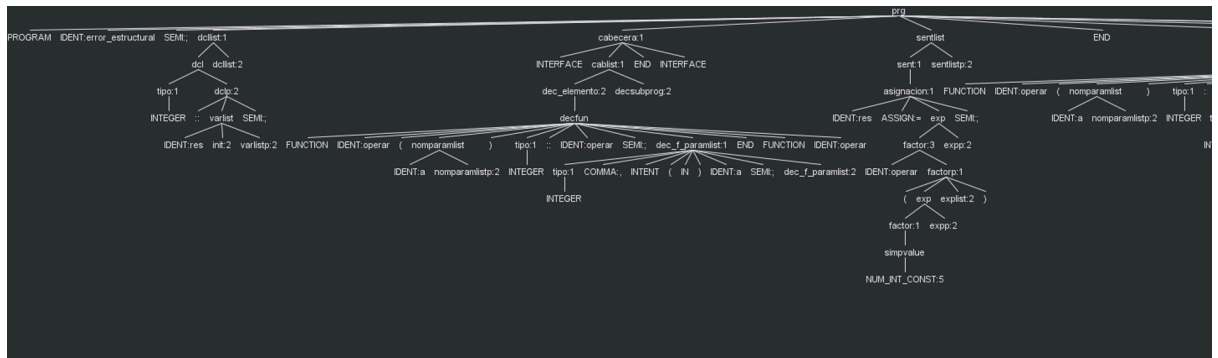
    operar = 0;

ENDIF

CALL log_error;

END FUNCTION operar
```

-Árbol Sintáctico:



-Error:

```
Iniciando análisis del archivo: PRUEBAS/CasoIncorrecto4.txt...

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 19 | Posición: 0
>> Detalle: mismatched input 'END' expecting {'(', ';'}
>> Token que falló: 'END'
#####

#####
>> ERROR SINTÁCTICO DETECTADO
>> Línea: 20 | Posición: 0
>> Detalle: mismatched input '<EOF>' expecting {'(', ';'}
>> Token que falló: '<EOF>'
#####

Análisis finalizado.
```

Los mensajes mismatched input 'END' y mismatched input '<EOF>' indican que el parser esperaba la asignación obligatoria de retorno y en su lugar encontró el fin de la función.

6. Anexo: Conjuntos Directores

A continuación se presentan los conjuntos directores:

```
DIR(prg -> 'PROGRAM' IDENT SEMI dcclist cabecera sentlist 'END'
'PROGRAM' IDENT subproglis EOF) = { PROGRAM }

DIR(dcclist -> dcl dcclist) = { INTEGER, REAL, CHARACTER }

DIR(dcclist -> lambda) = { INTERFACE, IDENT, CALL, IF, DO, SELECT
}

DIR(dcl -> tipo dclp) = { INTEGER, REAL, CHARACTER }

DIR(dclp -> COMMA 'PARAMETER' '::' IDENT ASSIGN simpvalue ctelist
SEMI) = { COMMA }

DIR(dclp -> '::' varlist SEMI) = { :: }

DIR(tipo -> 'INTEGER') = { INTEGER }

DIR(tipo -> 'REAL') = { REAL }

DIR(tipo -> 'CHARACTER' charlength) = { CHARACTER }

DIR(charlength -> '(' NUM_INT_CONST ')') = { ( }

DIR(charlength -> lambda) = { COMMA, ::, INTENT }

DIR(varlist -> IDENT init varlistp) = { IDENT }

DIR(varlistp -> COMMA IDENT init varlistp) = { COMMA }

DIR(varlistp -> lambda) = { SEMI }

DIR(init -> ASSIGN simpvalue) = { ASSIGN }

DIR(init -> lambda) = { COMMA, SEMI }

DIR(ctelist -> COMMA IDENT ASSIGN simpvalue ctelist) = { COMMA }

DIR(ctelist -> lambda) = { SEMI }

DIR(simpvalue -> NUM_INT_CONST | NUM_REAL_CONST | STRING_CONST |
NUM_INT_CONST_B | NUM_INT_CONST_O | NUM_INT_CONST_H | CONST_BOOL)
= { NUM_INT_CONST, NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B,
NUM_INT_CONST_O, NUM_INT_CONST_H, CONST_BOOL }

DIR(cabecera -> 'INTERFACE' cablist 'END' 'INTERFACE') = {
INTERFACE }
```

```

DIR(cabecera -> lambda) = { IDENT, CALL, IF, DO, SELECT }

DIR(cablist -> dec_elemento decsubprog) = { SUBROUTINE, FUNCTION }

DIR(cablist -> lambda) = { END }

DIR(decsubprog -> dec_elemento decsubprog) = { SUBROUTINE,
FUNCTION }

DIR(decsubprog -> lambda) = { END }

DIR(dec_elemento -> decproc) = { SUBROUTINE }

DIR(dec_elemento -> decfun) = { FUNCTION }

DIR(decproc -> 'SUBROUTINE' IDENT formal_paramlist dec_s_paramlist
'END' 'SUBROUTINE' IDENT) = { SUBROUTINE }

DIR(decfun -> 'FUNCTION' IDENT '(' nomparamlist ')' tipo '::'
IDENT SEMI dec_f_paramlist 'END' 'FUNCTION' IDENT) = { FUNCTION }

DIR(formal_paramlist -> '(' nomparamlist ')') = { ( }

DIR(formal_paramlist -> lambda) = { INTEGER, REAL, CHARACTER,
IDENT, CALL, IF, DO, SELECT }

DIR(nomparamlist -> IDENT nomparamlistp) = { IDENT }

DIR(nomparamlistp -> COMMA nomparamlist) = { COMMA }

DIR(nomparamlistp -> lambda) = { ) }

DIR(dec_s_paramlist -> tipo COMMA 'INTENT' '(' tipoparam ')' IDENT
SEMI dec_s_paramlist) = { INTEGER, REAL, CHARACTER }

DIR(dec_s_paramlist -> lambda) = { INTEGER, REAL, CHARACTER,
IDENT, CALL, IF, DO, SELECT }

DIR(dec_f_paramlist -> tipo COMMA 'INTENT' '(' 'IN' ')') IDENT SEMI
dec_f_paramlist) = { INTEGER, REAL, CHARACTER }

DIR(dec_f_paramlist -> lambda) = { INTEGER, REAL, CHARACTER,
IDENT, CALL, IF, DO, SELECT }

DIR(tipoparam -> 'IN' | 'OUT' | 'INOUT') = { IN, OUT, INOUT }

DIR(sentlist -> sent sentlistp) = { IDENT, CALL, IF, DO, SELECT }

DIR(sentlistp -> sent sentlistp) = { IDENT, CALL, IF, DO, SELECT }

DIR(sentlistp -> lambda) = { END, ENDIF, ELSE, ENDDO, CASE,
DEFAULT, IDENT }

```

```

DIR(sent -> asignacion) = { IDENT, CALL }

DIR(sent -> 'IF' '(' expcond ')' sentif) = { IF }

DIR(sent -> 'DO' sentdo) = { DO }

DIR(sent -> 'SELECT' 'CASE' '(' exp ')' casos 'END' 'SELECT') = {
SELECT }

DIR(asignacion -> IDENT ASSIGN exp SEMI) = { IDENT }

DIR(asignacion -> proc_call SEMI) = { CALL }

DIR(sentif -> sent) = { IDENT, CALL, IF, DO, SELECT }

DIR(sentif -> 'THEN' sentlist sentthen) = { THEN }

DIR(sentthen -> 'ENDIF') = { ENDIF }

DIR(sentthen -> 'ELSE' sentlist 'ENDIF') = { ELSE }

DIR(sentdo -> 'WHILE' '(' expcond ')' sentlist 'ENDDO') = { WHILE
}

DIR(sentdo -> IDENT ASSIGN doval COMMA doval COMMA doval sentlist
'ENDDO') = { IDENT }

DIR(doval -> NUM_INT_CONST) = { NUM_INT_CONST }

DIR(doval -> IDENT) = { IDENT }

DIR(casos -> 'CASE' casosp) = { CASE }

DIR(casos -> lambda) = { END }

DIR(casosp -> '(' etiquetas ')' sentlist casos) = { ( }

DIR(casosp -> 'DEFAULT' sentlist) = { DEFAULT }

DIR(etiquetas -> simpvalue etiquetasp) = { NUM_INT_CONST,
NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O,
NUM_INT_CONST_H, CONST_BOOL }

DIR(etiquetas -> COLON simpvalue) = { COLON }

DIR(etiquetasp -> listaetiquetas) = { COMMA, ) }

DIR(etiquetasp -> COLON simpvaluep) = { COLON }

DIR(listaetiquetas -> COMMA simpvalue listaetiquetas) = { COMMA }

DIR(listaetiquetas -> lambda) = { ) }

```

```

DIR(simpvaluep -> simpvalue) = { NUM_INT_CONST, NUM_REAL_CONST,
STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H,
CONST_BOOL }

DIR(simpvaluep -> lambda) = { } }

DIR(proc_call -> CALL IDENT subparamlist) = { CALL }

DIR(subparamlist -> '(' exp explist ')') = { ( }

DIR(subparamlist -> lambda) = { SEMI }

DIR(explist -> COMMA exp explist) = { COMMA }

DIR(explist -> lambda) = { ) }

DIR(exp -> factor exp) = { NUM_INT_CONST, NUM_REAL_CONST,
STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H,
CONST_BOOL, (, IDENT }

DIR(exp -> op factor exp) = { +, -, *, / }

DIR(exp -> lambda) = { SEMI, ), COMMA, THEN, ELSE, ENDIF, ENDDO,
CASE, DEFAULT, END, LT, GT, LE, GE, EQ, NE, .OR., .AND., .EQV.,
.NEQV. }

DIR(op -> oparit) = { +, -, *, / }

DIR(oparit -> '+' | '-' | '*' | '/') = { +, -, *, / }

DIR(factor -> simpvalue) = { NUM_INT_CONST, NUM_REAL_CONST,
STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O, NUM_INT_CONST_H,
CONST_BOOL }

DIR(factor -> '(' exp ')') = { ( }

DIR(factor -> IDENT factorp) = { IDENT }

DIR(factorp -> '(' exp explist ')') = { ( }

DIR(factorp -> lambda) = { +, -, *, /, SEMI, ), COMMA, THEN, ELSE,
ENDIF, ENDDO, CASE, DEFAULT, END, LT, GT, LE, GE, EQ, NE, .OR.,
.AND., .EQV., .NEQV. }

DIR(expcond -> factorcond expcondp) = { NUM_INT_CONST,
NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O,
NUM_INT_CONST_H, CONST_BOOL, (, IDENT, .NOT. }

DIR(expcondp -> oplog factorcond expcondp) = { .OR., .AND., .EQV.,
.NEQV. }

```

```

DIR(expcondp -> lambda) = { }, THEN, ELSE, ENDIF, ENDDO }

DIR(factorcond -> exp opcomp exp) = { NUM_INT_CONST,
NUM_REAL_CONST, STRING_CONST, NUM_INT_CONST_B, NUM_INT_CONST_O,
NUM_INT_CONST_H, CONST_BOOL, (, IDENT }

DIR(factorcond -> '(' expcond ')') = { ( }

DIR(factorcond -> '.NOT.' factorcond) = { .NOT. }

DIR(factorcond -> CONST_BOOL) = { CONST_BOOL }

DIR(oplog -> '.OR.' | '.AND.' | '.EQV.' | '.NEQV.') = { .OR.,
.AND., .EQV., .NEQV. }

DIR(opcomp -> LT | GT | LE | GE | EQ | NE) = { LT, GT, LE, GE, EQ,
NE }

DIR(subproglis -> codproc subproglis) = { SUBROUTINE }

DIR(subproglis -> codfun subproglis) = { FUNCTION }

DIR(subproglis -> lambda) = { EOF }

DIR(codproc -> 'SUBROUTINE' IDENT formal_paramlist dec_s_paramlist
dcclist sentlist 'END' 'SUBROUTINE' IDENT) = { SUBROUTINE }

DIR(codfun -> 'FUNCTION' IDENT '(' nomparamlist ')' tipo '::'
IDENT SEMI dec_f_paramlist dcclist sentlist IDENT ASSIGN exp SEMI
'END' 'FUNCTION' IDENT) = { FUNCTION }

```